Spec Kit v1.2.5 · Kiro

Spec-Driven Development

Deja de pedirle a la IA que adivine: dale una spec que pueda seguir.



Chamba Lab · 2026



La spec manda

«El código sirve a la especificación, no al revés.»

- Primero escribes **qué** y **por qué**; el cómo llega después
- El agente implementa desde documentos, no desde un prompt suelto
- Las dudas aparecen en la spec, no en producción

Anatomía: las partes de un spec.md

`Escenarios` · `Requisitos` · `Éxito`

User Scenarios — Requirements — Success Criteria

Escenarios

Historias con prioridad y su prueba `Given / When / Then`: «cancela con 24 h → recibe el 100 %».

Requisitos

Lo que el sistema DEBE hacer: `FR-001` «permitir cancelar hasta 1 h antes del partido».

Éxito

Una métrica que se mide: `SC-001` «el jugador cancela en menos de 30 s».

Niveles: cuánto manda la spec

Vibe coding

Un prompt en el chat y a ver qué sale. Sin documento.

`"agrega cancelación"`

EL PUNTO DE PARTIDA

Spec-first

La spec se escribe antes de la tarea y guía al agente.

`Spec Kit` `Kiro`

EL MÁS COMÚN

Spec-anchored

La spec se conserva y evoluciona con la funcionalidad.

`Tessl`

EN CRECIMIENTO

Spec-as-source

Solo se edita la spec; nadie toca el código a mano.

`Tessl (explora)`

EXPERIMENTAL

Actores: quién hace qué



- **Tú:** decides el qué y apruebas cada documento
- **El agente** (Claude Code, Copilot): redacta y programa
- **Spec Kit:** CLI, plantillas y comandos
- **La constitución:** reglas que todo plan respeta

```
canchaya/
├─ .specify/
│  └─ memory/constitution.md
├─ .claude/skills/speckit-*/
└─ specs/
   └─ 001-cancelar-reserva/
      ├─ spec.md
      ├─ plan.md
      ├─ research.md
      ├─ data-model.md
      ├─ contracts/
      └─ tasks.md
```

Una carpeta numerada por funcionalidad, con todos sus documentos.

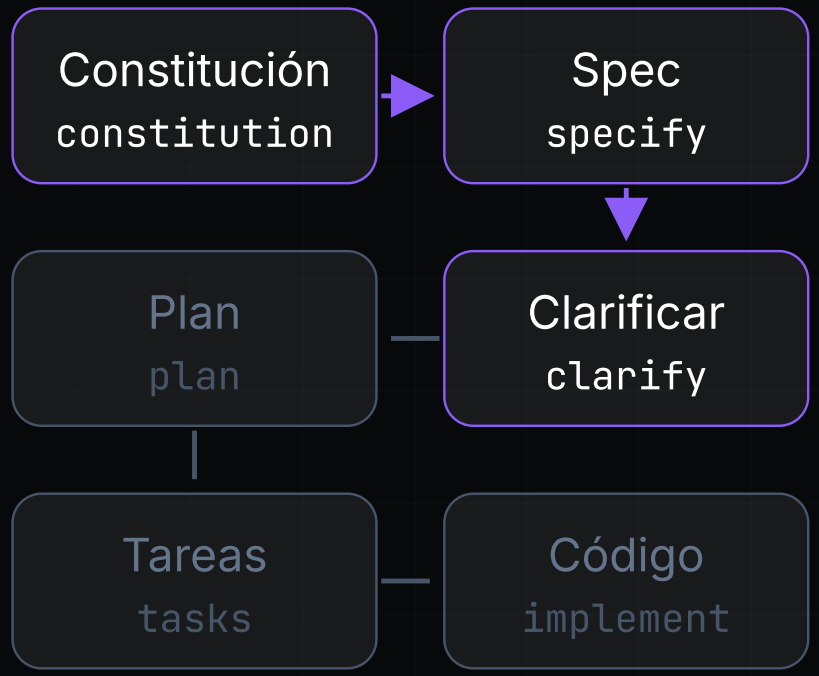


Una cadena de documentos

Cada paso deja un archivo que alimenta al siguiente.

- **Constitución:** las reglas de todo el proyecto
- **Spec:** qué y por qué, sin tecnología
- **Plan y tareas:** cómo y en qué orden
- **Código:** la expresión de la spec

Paso 1: reglas, spec y dudas




```
/speckit-constitution Código en
  TypeScript, tests primero y
  montos siempre en céntimos

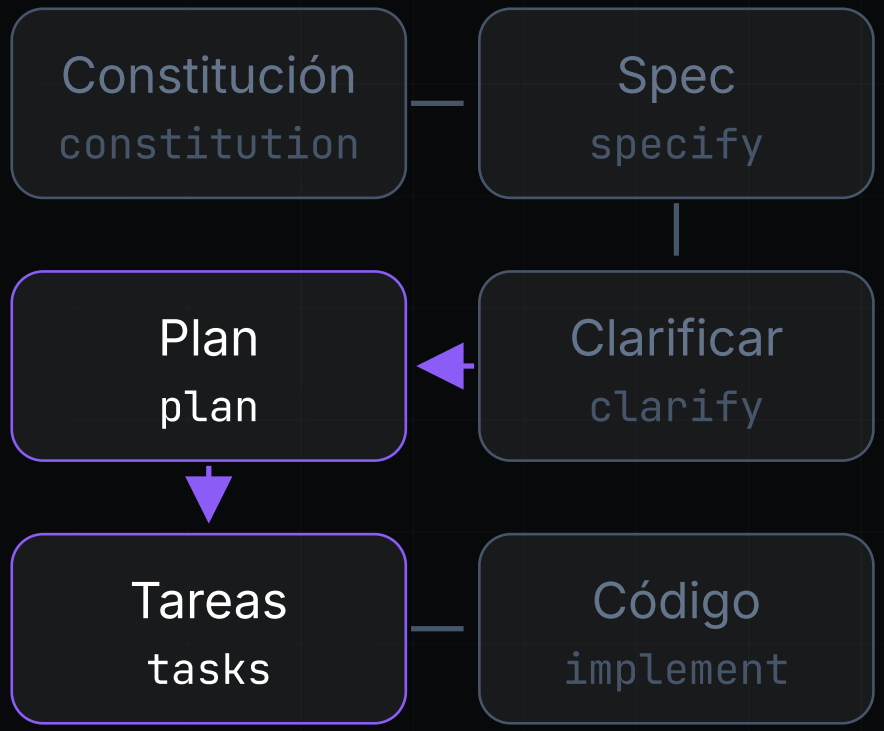
/speckit-specify Permitir cancelar
  una reserva de cancha y devolver
  el pago según la anticipación

/speckit-clarify
```

Describe el qué y el porqué. Ni una palabra de tecnología.

- `clarify` te hace hasta 5 preguntas antes de planificar

Paso 2: el plan y las tareas




```
/speckit-plan Next.js con API
  routes, PostgreSQL con Prisma
  y Stripe para los reembolsos

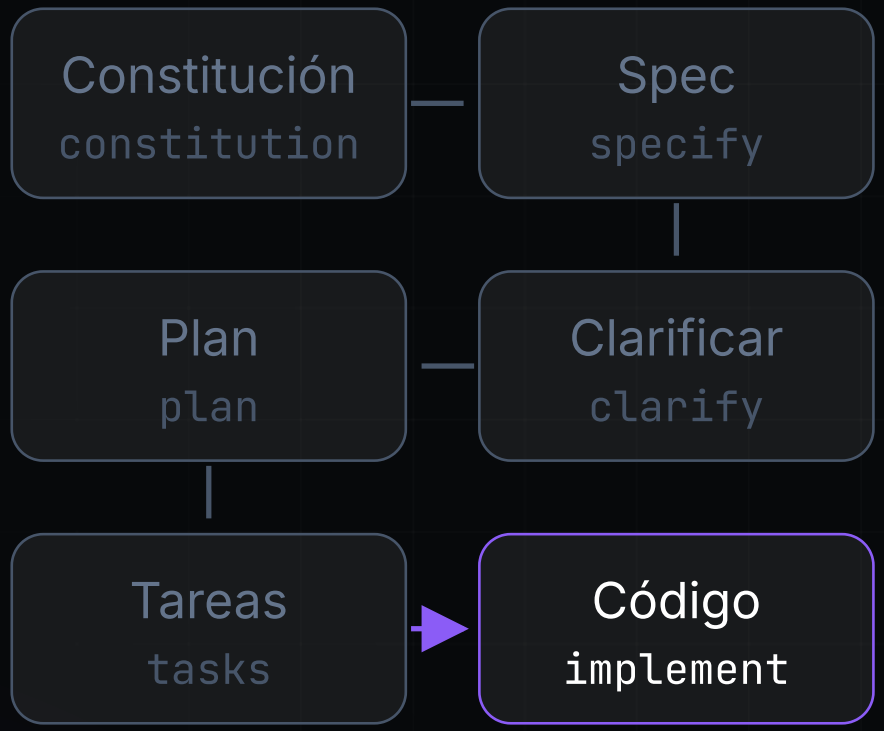
/speckit-tasks

/speckit-analyze
```

Aquí, y solo aquí, entra la tecnología.

- El plan deja `research.md`, `data-model.md` y `contracts/`

Paso 3: construir y comprobar




```
/speckit-implement
# sigue tasks.md fase por fase

/speckit-converge
# revisa el código contra spec,
# plan y tareas
```

El agente comprueba lo que construyó contra lo que prometió.

- converge cierra el ciclo: contrasta el código con la spec

Setup: Spec Kit en Claude Code

terminal

```
# 1. Instalar la CLI (Python 3.11+ y uv)
uv tool install specify-cli

# 2. Crear el proyecto para Claude Code
specify init canchaya --integration claude

# 3. Comprobar la instalación
specify check
```

Instala los comandos como skills en `.claude/skills/speckit-<comando>/`; se invocan con `/speckit-specify`.

¿Proyecto que ya existe? `specify integration install claude`.

Otro agente, otra clave: `--integration copilot`.

Regla 1: la constitución del proyecto

.specify/memory/constitution.md

```
# CanchaYa Constitution

## Core Principles

### I. Tests primero
Todo requisito FR tiene su test antes del código.

### II. Dinero exacto
Los montos se guardan en céntimos, nunca en float.

### III. Simplicidad
Ninguna dependencia nueva sin justificarla en plan.md.

## Governance
La constitución manda sobre cualquier plan.
```

Se escribe una vez y vale para todas las funcionalidades.

Cada `/speckit-plan` la lee y debe respetarla.

Pocos principios y verificables: «tests primero», no «código bonito».

Regla 2: marca lo que no sabes



- Cada requisito, con su ID y un **MUST** comprobable
- La duda no se adivina: se marca
- No hay plan mientras quede un marcador

```
### User Story 1 - Cancelar (P1)
1. **Given** una reserva pagada,
   **When** se cancela con 24 h,
   **Then** se devuelve el 100 %.

- **FR-001**: System MUST permitir
  cancelar hasta 1 h antes.
- **FR-002**: System MUST devolver
  el pago completo con 24 h o más.
- **FR-003**: System MUST devolver
  [NEEDS CLARIFICATION: ¿cuánto
  si faltan menos de 24 h?]
```

El agente deja la pregunta escrita en lugar de inventar la respuesta.

Regla 3: tareas por fases y en paralelo

specs/001-cancelar-reserva/tasks.md

```
## Phase 2: Foundational
- [ ] T004 Agregar estado CANCELADA a Reserva
- [ ] T005 [P] Configurar cliente de Stripe

## Phase 3: User Story 1 - Cancelar (P1)
- [ ] T006 [P] [US1] Test de política de reembolso
- [ ] T007 [P] [US1] Test de cancelación tardía
- [ ] T008 [US1] calcularReembolso en src/reembolso.ts
- [ ] T009 [US1] POST /api/reservas/[id]/cancelar
```

Formato fijo: `ID [P?] [Historia] Descripción.`

`[P]`: archivos distintos y sin dependencias, se puede hacer en paralelo.

Una fase por historia de usuario, en orden de prioridad: P1 es el MVP.

Variantes: Spec Kit frente a Kiro

Spec Kit · GitHub

```
# spec.md · Given / When / Then
**Given** una reserva pagada,
**When** se cancela con 24 h,
**Then** se devuelve el 100 %.
```

CLI open source que se instala en tu agente: Claude Code, Copilot y más

Kiro · AWS

```
# requirements.md · EARS
WHEN un jugador cancela con 24 h
THE SYSTEM SHALL devolver el 100 %
```

IDE propio: requisitos → diseño → tareas, o design-first si manda la arquitectura

El dolor: el reembolso que nadie especificó



Prompt: «agrega cancelación de reservas».
El agente no preguntó: adivinó. Devuelve el 100 % a 10 minutos del partido y borra el historial.

```
// generado sin spec
export async function cancelarReserva(id: string) {
  const reserva = await db.reserva.findUnique({
    where: { id },
  });
  await stripe.refunds.create({
    payment_intent: reserva.pagoId,
    // sin amount: Stripe devuelve todo
  });
  await db.reserva.delete({ where: { id } });
}
```

Matriz de decisión: ¿prompt o spec?

	Vibe coding	Spec Kit	Kiro
Cambio pequeño	Ideal: un texto, un bug obvio	Excesivo	Quick Spec lo agiliza
Reglas de negocio	El agente adivina	Ideal: clarify antes del plan	Ideal: EARS y diseño
Tu herramienta	Cualquiera	La que ya usas	Su IDE o su CLI
Rastro en el repo	Ninguno	spec, plan, tareas y converge	requisitos, diseño y tareas

¿Una línea? Prompt directo. ¿Una funcionalidad con reglas, como la cancelación de CanchaYa? Spec Kit en el agente que ya usas; Kiro solo si tu equipo ya trabaja en su IDE.

Lo que te llevas

- Primero qué y por qué; la tecnología entra en el plan
- Una duda marcada en la spec vale más que un bug en producción
- Empieza spec-first con una funcionalidad que tenga reglas

> docs: github.com/github/spec-kit

> docs: kiro.dev/docs/specs

> niveles SDD: martinfowler.com · Birgitta Böckeler

> repo: github.com/Chamba-Lab